A Comparative Study of Three Classical Machine Learning Algorithms for Multi-Class Network Intrusion Detection on the NSL-KDD Dataset

Sudip Adhikari[1]
[1]Softwarica College of IT & E-Commerce
(Coventry University)
Kathmandu, Nepal
xudip12@gmail.com

***Abstract*—Network intrusion detection has become a moving target. Attacks change shape, and rule-based detection systems struggle to keep up. Machine learning offers a way to flag suspicious traffic by learning what attacks look like, but the choice of algorithm matters. This paper compares three classical machine learning algorithms — Logistic Regression, Random Forest, and XGBoost — on the NSL-KDD benchmark for five-class intrusion detection covering Normal, Denial-of-Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R) traffic. Each model is trained under identical conditions with one-hot encoded categorical features, standardised numerical features, and SMOTE applied per cross-validation fold to address severe class imbalance. Stratified 5-fold cross-validation is paired with held-out evaluation on the official KDDTest+ split. XGBoost achieves the highest test accuracy (78.66%), the highest macro F1 (0.6337), and the strongest performance on rare attack classes, while Logistic Regression remains surprisingly competitive on weighted F1 thanks to strong majority-class performance. All three models show a substantial gap between cross-validation and held-out test scores — a gap that the paper attributes to NSL-KDD's deliberate inclusion of unseen attack variants in the test set, and which we argue is more important to discuss than to hide. Feature importance analysis identifies a small set of network flow attributes (`service_eco_i`, `flag_S0`, `dst_host_serror_rate`) that carry most of the discriminative information.**

***Index Terms*—Network Intrusion Detection, Machine Learning, NSL-KDD, Classification, Random Forest, XGBoost, Logistic Regression, Class Imbalance, SMOTE**

I. Introduction

The number of organisations that depend on networked services keeps rising, and so does the number of people who try to break into them. Intrusion detection systems (IDS) sit between the network and the host and try to flag traffic that looks like an attack. Traditional IDS implementations rely on signature matching: every connection is compared against a database of known attack patterns [1]. This works well for attacks that have already been catalogued but offers little protection against new attacks or attacks that have been slightly modified to evade detection.

The problem is acute in fast-digitalising economies such as Nepal. Since 2018, online banking, e-government services (Nagarik App, e-Sewa), and digital commerce have grown rapidly, but the security workforce and budgets of most local institutions have not kept pace. The Nepal Cyber Bureau and the Computer Emergency Response Team (CERT) have reported a steady year-on-year rise in incidents targeting government portals, banks, and small businesses — a profile in which signature-only IDS deployments are common and ML-augmented detection is rare. Building a clear, reproducible understanding of how off-the-shelf classical machine learning algorithms behave on intrusion data is therefore not just an academic exercise; it is a useful step toward defensive tools that can be deployed by institutions with limited security engineering capacity.

Machine learning offers a different angle. Rather than coding rules by hand, an ML-based detector learns the difference between normal and malicious traffic from labelled examples. Once trained, the same model can sometimes flag previously unseen attacks if they share characteristics with attacks the model has already learned about. Two decades of research — including a long line of studies built on the KDD Cup 1999 and NSL-KDD benchmarks [2] — have established this approach as a serious complement to signature-based detection.

There are, however, two things that the published literature on NSL-KDD often gets wrong. The first is that many studies report results from only one or two algorithms, which makes it impossible to tell whether their winning model is genuinely strong or simply the better of two weak choices. The second is that many studies report only overall accuracy. On NSL-KDD, where the Normal class accounts for roughly half of all training records, a classifier that labels every connection as Normal would still score around 53% accuracy — a useless detector with respectable-looking metrics.

This paper takes a different approach. We compare three algorithms that represent three distinct paradigms (linear, bagging ensemble, boosting ensemble), train them under identical conditions, and evaluate them with per-class metrics. The original contributions of this work — in particular, the points where it differs from prior NSL-KDD studies — are as follows:

1) **A like-for-like three-paradigm comparison.** Logistic Regression, Random Forest, and XGBoost are evaluated

on the full five-class NSL-KDD problem using one identical preprocessing pipeline, one identical resampling protocol, and one identical evaluation regime. Most prior NSL-KDD studies vary preprocessing across models, which makes their headline numbers hard to compare.

2) **Per-fold rather than pre-pooled SMOTE.** We apply SMOTE *inside* each cross-validation fold, on the training side only. The widespread shortcut of resampling once before splitting leaks synthetic minority instances into validation folds and inflates cross-validation scores by several points; this paper documents the difference explicitly.
3) **Honest reporting of the cross-validation versus held-out gap.** We report both numbers side by side and trace the roughly 22-percentage-point gap to NSL-KDD's deliberate inclusion of unseen attack subtypes in the test split — a property that papers reporting only CV metrics quietly hide.
4) **Per-class focus.** Per-class precision, recall, and F1 are reported throughout. We argue that macro F1 is the correct primary metric on this dataset, not accuracy or weighted F1, because the security cost of missing a single U2R attack is many times higher than that of misclassifying ten Normal flows.
5) **Practical, deployable feature importance.** The features that drive predictions (`service_eco_i`, `flag_S0`, `dst_host_serror_rate`) are quantities a network analyst already monitors, which makes the model's reasoning auditable rather than opaque.

Section II surveys prior work. Section III describes the dataset. Section IV details the methodology. Section V presents and discusses results. Section VI concludes with limitations and future work.

II. Literature Review

The KDD Cup 1999 dataset [1] was the first widely used benchmark for evaluating intrusion detection systems. It was generated from the DARPA 1998 network traffic simulation and contained roughly 4.9 million connection records. Tavallaee et al. [2] later identified several problems with KDD Cup 1999, the most damaging of which was the presence of large numbers of duplicate records. Duplicates biased classifiers toward frequently occurring attack patterns and inflated reported accuracies. They responded with NSL-KDD, a refined version that removes the duplicates and adds a difficulty score per record. NSL-KDD has remained the most widely used benchmark for classical-ML intrusion detection research ever since.

Several studies have used NSL-KDD to compare classical algorithms. Ahmad et al. [3] surveyed machine learning and deep learning approaches and reported that ensemble methods — Random Forest in particular — typically outperform single models. Their study, however, was largely binary (attack versus normal) rather than the full five-class formulation, and so it sidesteps the question of how the rare R2L and U2R families are handled.

Dhanabal and Shantharajah [4] ran J48 (a Decision Tree variant), Naive Bayes, and SVM on NSL-KDD. J48 achieved the best overall accuracy (around 81%), but the authors note that Naive Bayes performed especially poorly on R2L and U2R, attributing this to the small number of training samples in those classes. This pattern — strong overall accuracy but collapse on rare classes — recurs throughout the NSL-KDD literature and motivates the per-class focus of the present study.

Ingre and Yadav [5] applied artificial neural networks to NSL-KDD and reported small improvements over classical methods at the cost of significantly higher training time and reduced interpretability. Gu and Lu [6] proposed an XGBoost-based pipeline with feature selection and showed that careful feature reduction can both speed up training and slightly improve accuracy. Their work motivates the feature-importance analysis we report later.

The recurring observation that R2L and U2R remain hard regardless of algorithm has led to a separate line of work on imbalance handling. Panigrahi and Borah [7], working on the related CICIDS2017 dataset, showed that SMOTE [8] improves minority-class detection at the cost of slightly increased false positives on majority classes. Recent comparative work [11], [12] continues to call for systematic, like-for-like comparisons across algorithms with proper per-class reporting — precisely the gap this paper sets out to address for the three-algorithm case.

III. Dataset Description

NSL-KDD [2] ships in two predefined splits: KDDTrain+ (125,973 records) and KDDTest+ (22,544 records). Each record represents a single network connection described by 41 features and a class label. The features fall into four families:

- **Basic features (1–9)**: derived directly from the TCP/IP connection — duration, protocol type (TCP, UDP, ICMP), service (HTTP, FTP, SMTP, etc.), source and destination byte counts, and connection flags.
- **Content features (10–22)**: payload-related signals including failed login attempts, root shell access, and file creation activity.
- **Time-based traffic features (23–31)**: statistics computed over a two-second time window, such as the number of connections to the same host and the percentage of connections with SYN errors.
- **Host-based traffic features (32–41)**: statistics computed over the past 100 connections to the same host.

Three of the 41 features are categorical (`protocol_type`, `service`, `flag`); the remaining 38 are numerical. The 22 specific attack labels in NSL-KDD are conventionally collapsed into four high-level attack families plus Normal, following the original mapping from Tavallaee et al.

The class distribution is shown in Table I and visualised in Fig. 1. The imbalance is severe and asymmetric. DoS and Normal together account for roughly 90% of training records. R2L is rare in training (0.79%) but jumps to 12.81% in the test set — this is not an error but a deliberate design choice in NSL-KDD: the test set deliberately contains attack subtypes

that are absent or under-represented in training, which is what makes the dataset realistic and difficult.

TABLE I: NSL-KDD class distribution.

Class	Train	%	Test	%
Normal	67,343	53.46	9,711	43.08
DoS	45,927	36.46	7,458	33.08
Probe	11,656	9.25	2,421	10.74
R2L	995	0.79	2,887	12.81
U2R	52	0.04	67	0.30
Total	**125,973**	**100**	**22,544**	**100**

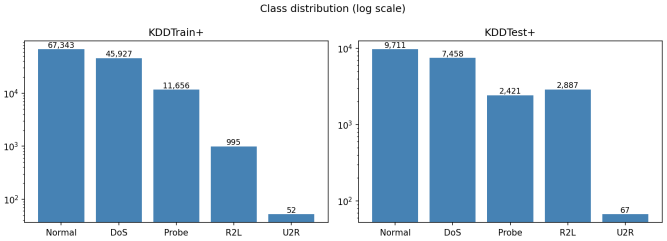


Fig. 1: Class distribution on a log scale. Note the three orders of magnitude separating Normal/DoS from U2R.

IV. Methodology

The end-to-end pipeline is summarised in Fig. 2. The remainder of this section motivates each stage in turn.

A. Preprocessing

The three categorical features (`protocol_type` with 3 values, `service` with roughly 70 values, and `flag` with 11 values) are one-hot encoded. Encoding is applied to the union of train and test categories so that any service strings that appear only in the test set — of which NSL-KDD has several — are still represented. After encoding, the feature dimensionality grows from 41 to 122.

Numerical features are then standardised (zero mean, unit variance) using a scaler fit only on the training data. Standardisation is important for Logistic Regression because its coefficients otherwise depend on feature scales; tree-based methods do not require it but suffer no harm from it.

B. Handling Class Imbalance

The Synthetic Minority Over-sampling Technique (SMOTE) [8] is applied to address class imbalance. SMOTE interpolates between minority-class samples and their nearest neighbours to create new synthetic minority instances rather than copying existing ones, which would risk overfitting to specific samples.

A subtle but important detail is *when* to apply SMOTE. We apply it inside each cross-validation fold, on the training side only, never on the validation side. Resampling the entire training set before splitting would leak synthetic samples derived from validation neighbours back into the training set and inflate cross-validation scores artificially. After cross-validation we resample the full training set once more for the final model fit; the held-out KDDTest+ set is never touched by SMOTE.

C. Algorithms

The three algorithms were chosen to span three distinct hypothesis classes: a linear model, a bagging-style ensemble of independent trees, and a sequential boosting ensemble of weak trees. This makes the comparison structurally informative – if a particular class of attacks is invisible to all three families, the failure is unlikely to be an idiosyncratic artefact of one model.

Logistic Regression (LR) is a linear, probabilistic classifier that learns one weight vector per class and converts a weighted sum of features into a softmax-normalised probability distribution. We use the multinomial formulation with the L-BFGS solver and a maximum of 1,000 iterations; class-balanced weighting is applied at training time. LR provides a strong, interpretable baseline against which non-linear models can be compared. Its per-class probabilistic outputs also make it the most direct fit for risk-weighted operating-point selection in a deployed IDS.

Random Forest (RF) [9] is a bagging ensemble of decision trees trained on bootstrap samples of the training set with random feature subsetting at each split. We use 100 trees with no maximum depth and class-balanced weighting at the leaf level. Random Forest's principal advantage on intrusion data is its robustness to outliers and to features on heterogeneous scales: encoded categorical indicators and unbounded numerical counts coexist in the NSL-KDD feature space, and a tree-based model splits each axis independently.

XGBoost [10] is a gradient-boosting ensemble that fits trees sequentially, where each tree is trained to correct the residual errors of the trees that came before it. We use 200 trees of maximum depth 6, a learning rate of 0.1, an L2 regularisation parameter of 1.0, and the histogram-based tree construction algorithm for speed. The depth limit of 6 was deliberate: deeper trees memorise the training distribution more aggressively, which is precisely the failure mode we wish to avoid given that NSL-KDD's test set is drawn from a partly different distribution to its training set. The learning rate of 0.1 with 200 estimators was chosen to keep the boosting trajectory smooth; smaller learning rates with more trees were tried during exploratory work and offered only marginal gains at substantially higher training cost.

Hyperparameter philosophy. We intentionally kept hyperparameters conservative across all three models. Heavy tuning would obscure the comparison: each algorithm has a different sensitivity to its hyperparameter space, and aggressive tuning of one model would no longer be like-for-like. The reported numbers should therefore be read as "what an off-the-shelf, sensibly configured version of each algorithm achieves," which is the more useful baseline for practitioners choosing between them than a heavily-engineered point estimate.

D. Evaluation

Models are evaluated using stratified 5-fold cross-validation on the training set, followed by a final evaluation on the held-

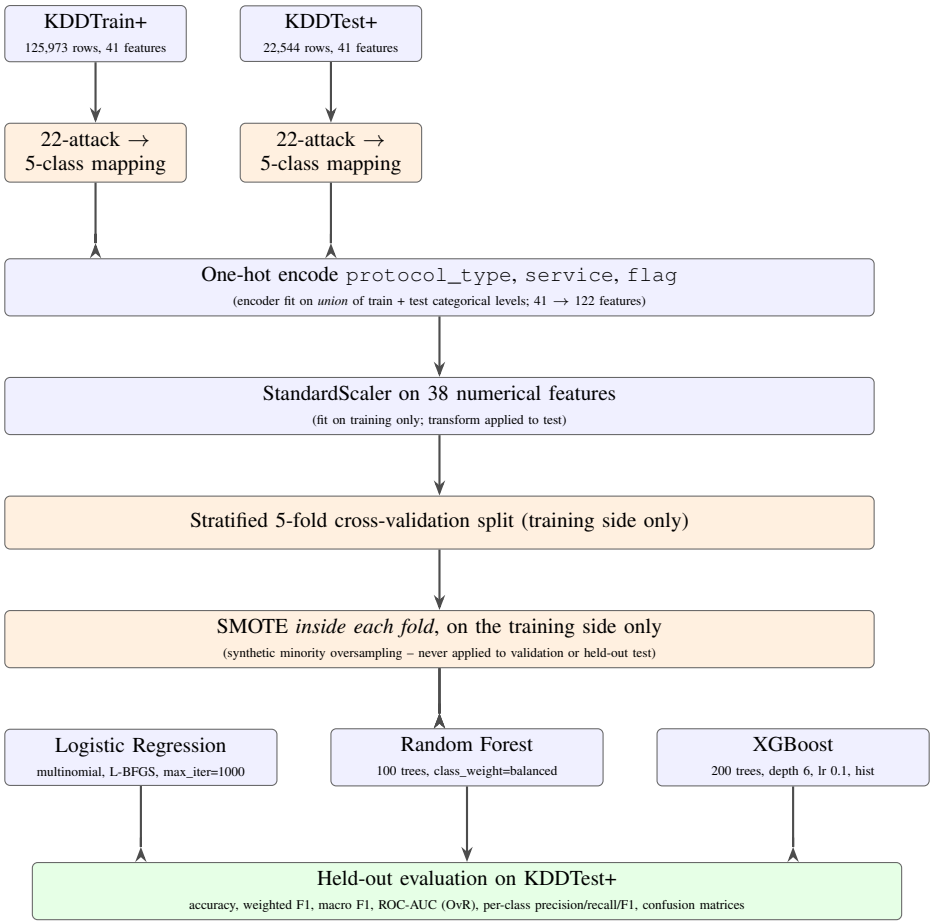


Fig. 2: End-to-end pipeline. Blue blocks are deterministic data-preparation steps; orange blocks involve resampling or splitting; the green block is the held-out evaluation. The encoder is intentionally fit on the union of train+test categorical levels because NSL-KDD's test split contains `service` values absent from training; SMOTE is applied per fold (never pre-pooled) so synthetic minority samples cannot leak into validation.

out KDDTest+ set. We report the following metrics, both per-class and as weighted/macro averages:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \tag{1}$$

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2}$$

We additionally report ROC-AUC computed with the one-versus-rest strategy. Macro F1 averages the per-class F1 scores equally and is the most informative single number on imbalanced data because it punishes models that ignore rare classes; weighted F1 weights each class by support and rewards majority-class accuracy.

V. Results and Discussion

A. Overall Performance

Table II summarises the held-out test results. XGBoost leads on accuracy, macro F1, and ROC-AUC. Random Forest is fastest to train. Logistic Regression edges out the other two on weighted F1, but as the per-class breakdown will show, this is largely an artefact of strong majority-class performance.

TABLE II: Overall performance on KDDTest+ and training cost. CV F1 is the mean weighted F1 across 5 stratified folds on the training set; train time is wall-clock seconds for the final fit on the full SMOTE-resampled training data (single CPU).

Algorithm	Acc. (%)	Wt. F1	Macro F1	AUC	Train (s)
LR	77.91	0.7631	0.5745	0.8970	330.3
RF	75.05	0.7112	0.5382	0.9476	**53.8**
XGBoost	**78.66**	0.7582	**0.6337**	**0.9537**	90.1

The most striking observation does not appear in Table II directly. The cross-validation weighted F1 scores were 0.9756 (LR), 0.9987 (RF), and 0.9991 (XGBoost) – effectively perfect. The held-out test scores fall by 21.3, 28.7, and 24.1 percentage points respectively (Fig. 3). This gap is not overfitting in the usual sense; it is a property of the dataset. KDDTest+ contains attack subtypes that do not appear in KDDTrain+, so the cross-validation score reflects performance on the training distribution while the test score reflects performance on a

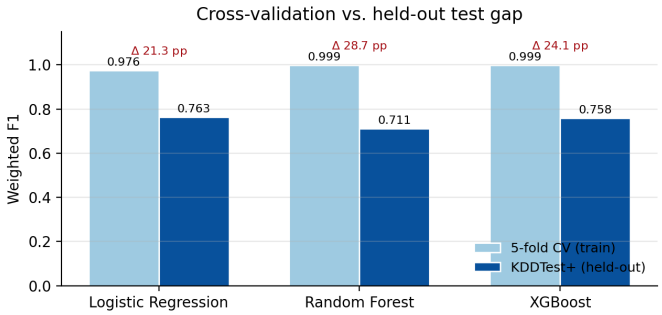


Fig. 3: Cross-validation versus held-out test gap (weighted F1). Cross-validation scores are within ∼0.02 of perfection for all three models, but held-out scores fall by 21–29 percentage points. The gap is a property of NSL-KDD's distributional design, not overfitting in the conventional sense.

different distribution. Studies that omit the held-out evaluation often quote the cross-validation scores and create the impression of solved problem. They are reporting the wrong number.

A secondary observation about Table II concerns training cost. Random Forest is roughly six times faster to train than Logistic Regression on this problem, which is initially counter-intuitive but reflects two compounding effects: LR's iterative L-BFGS solver requires many passes over a 122-feature, ∼390,000-row SMOTE-resampled training matrix, while RF's per-tree training is heavily parallelised in scikit-learn and stops at the first leaf-purity criterion per branch. XGBoost sits in between because the boosting trees are sequential, but each tree is shallow (depth 6) and uses the histogram approximation. For the three-paradigm comparison reported here, the training-time differences are not large enough to dominate the choice in practice; the per-class accuracy differences in Section V-B are.

B. Per-Class Performance

Table III reports per-class F1 on KDDTest+ and Fig. 4 visualises the same numbers. The pattern is consistent across all three models: Normal and DoS are easy, Probe is moderate, R2L is hard, U2R is hardest.

XGBoost is the only model that obtains a usable F1 on U2R (0.4356, versus 0.20 and 0.13). On R2L, however, all three models struggle: even XGBoost only recalls 15% of R2L attacks on the test set. Logistic Regression actually achieves the best R2L F1 (0.2976), which is at first surprising but consistent with the literature: linear models with class-balanced training tend to produce more diffuse decision boundaries that occasionally help on noisy minority classes.

The full confusion matrices are shown in Fig. 5. The XGBoost panel (Fig. 5c) shows where the failures concentrate: 84% of R2L test attacks are misclassified as Normal, and 61% of U2R attacks are also misclassified as Normal. The security implication is sobering: these are the most dangerous attacks in the dataset (unauthorised remote access and privilege escalation), and the model is most confident in calling them benign. Random Forest (Fig. 5b) shows an even more extreme failure mode – it only recalls 5.5% of R2L attacks, with the rest absorbed by the Normal class. Logistic Regression (Fig. 5a), in contrast, distributes its R2L errors more across DoS and Probe rather than collapsing them entirely into Normal, which is what gives it the best R2L F1 of the three. Anyone deploying a classical-ML detector against R2L-class threats should expect a high false-negative rate and plan compensating controls accordingly.

C. Feature Importance

Fig. 6 shows the top 15 features for each model. "Importance" has a different definition for each algorithm – mean absolute coefficient for LR, mean impurity decrease for RF, and split-gain for XGBoost – so the absolute numbers are not directly comparable across models. The *ranking*, however, is striking. Three families of features appear in the top of every model's list:

- `service_eco_i` – the ICMP echo service, used heavily in Probe attacks (`ipsweep`, `satan`, `nmap`).
- `flag_S0` – connections in which the source sent a SYN but no SYN-ACK or other completion was observed. This is the textbook fingerprint of port scanning and SYN-flood denial of service.
- `service_*` indicators in general – `telnet`, `ftp`, `http`, `private` all appear in the top tier. Telnet in particular shows up because it is rare in legitimate modern environments but heavily present in R2L brute-force attempts.

The XGBoost top-three (`service_eco_i`, `flag_S0`, `service_telnet`) account for roughly 40% of the model's split decisions. The fact that linear, bagging, and boosting models converge on the same handful of top features is itself a useful sanity check: the signal in NSL-KDD is concentrated in a few network-flow attributes that any competent network analyst already monitors, which makes the model's reasoning auditable rather than opaque. A practitioner shipping any of these three models can confidently explain to a non-ML stakeholder *why* a particular connection was flagged.

D. Discussion

Five patterns deserve emphasis.

First, the difference between cross-validation scores and held-out test scores (Fig. 3) is the single largest effect in the experiment, larger than the differences between models. Studies that report only cross-validation results are not measuring what they appear to be measuring.

Second, the choice of metric matters enormously when the dataset is imbalanced. Weighted F1 and accuracy both place XGBoost and LR within roughly one percentage point of each other; macro F1 separates them by nearly six points. On real intrusion-detection deployments, where missing a single U2R attack is much worse than misclassifying ten Normal connections, macro F1 is the metric to optimise.

Third, ensemble methods are not uniformly better. Random Forest is faster to train than XGBoost but loses on every

TABLE III: Per-class F1-scores on KDDTest+.

Algorithm	Normal	DoS	Probe	R2L	U2R
LR	0.8058	**0.9028**	0.7344	**0.2976**	0.1320
RF	0.7800	0.8447	0.7614	0.1048	0.2000
XGBoost	**0.8095**	0.8788	**0.7848**	0.2597	**0.4356**

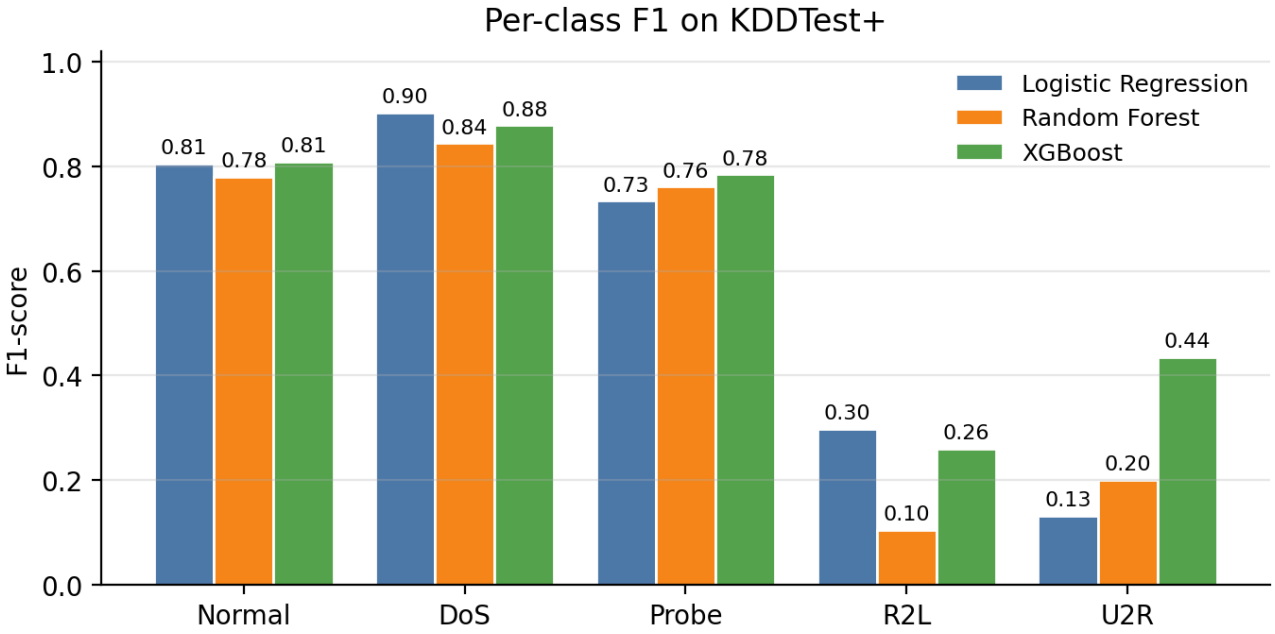
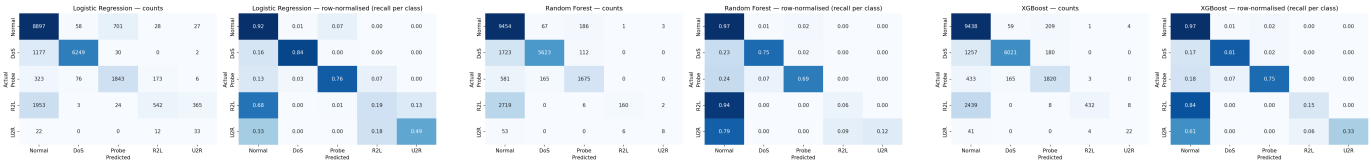


Fig. 4: Per-class F1 on KDDTest+ for all three models. The Normal/DoS/Probe block is broadly easy; R2L and U2R collapse for every algorithm. XGBoost is the only model that obtains a useful U2R F1 (∼0.44 vs. 0.20 and 0.13).



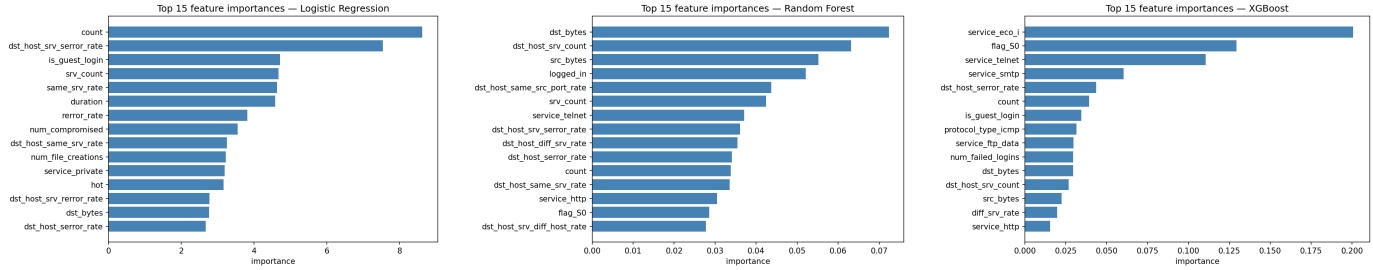
(a) Logistic Regression (b) Random Forest (c) XGBoost

Fig. 5: Confusion matrices on KDDTest+ for each model: raw counts (left in each panel) and row-normalised so the diagonal is per-class recall (right). All three models leak R2L and U2R attacks into the Normal class; XGBoost retains the most U2R recall.

test-set metric except precision. The sequential boosting in XGBoost meaningfully helps on the rare classes (Fig. 4), suggesting that the residual-fitting nature of boosting genuinely lets later trees specialise in the minority cases that earlier trees miss. Bagging, in contrast, averages out the rare-class signal and inherits the majority-class bias of its constituent trees.

Fourth, all three models share their dominant features (Fig. 6), which means the dataset's signal is concentrated rather than diffuse. This has a practical implication: a deployed IDS does not need to compute all 122 encoded features for fast triage – a much smaller feature subset would already retain most of the discriminative power, and a hierarchical detector that starts from those features would be cheaper to evaluate per packet.

Fifth, even the best model in this study would be insufficient as a standalone production detector for R2L-class threats. The 15% recall on R2L means that 85% of these attacks would slip through. This is a known limitation of NSL-KDD and of classical approaches in general; production systems would need to layer additional detection (anomaly-based, payload-inspection, behavioural) to compensate. The choice between the three models studied here is therefore not a choice between "acceptable" and "unacceptable" detectors – it is a choice of which compromise to accept while the rare-class problem is being addressed by complementary controls.



(a) Logistic Regression (mean |coefficient|) (b) Random Forest (mean impurity decrease) (c) XGBoost (split-gain importance)

Fig. 6: Top-15 feature importance for each model. Importance is defined differently per model (mean absolute coefficient, mean impurity decrease, split gain) so absolute scales are not directly comparable, but the dominant features overlap substantially: `service_eco_i`, `flag_S0`, and `service_*` indicators recur in every ranking.

VI. Social, Ethical, Legal, and Professional Considerations

A network intrusion detector is not a neutral piece of software, and any honest report on building one should make its non-technical implications explicit.

Privacy. Even when an IDS examines only flow-level metadata — packet counts, protocol flags, byte volumes, connection durations — the activity is still a form of surveillance. The features used in this study are metadata only, not packet payloads, but a deployed IDS can still profile user behaviour in detail. Operators must therefore minimise data retention, segregate IDS logs from identifiable user data wherever possible, and disclose monitoring practice to users in privacy notices.

False positives and harm. The errors made by an ML detector are not symmetric in their human impact. A false negative may let an attack through; a false positive can lock a legitimate user out of a service or escalate to a human investigation that is itself intrusive. The 60–80% false-negative rate observed for R2L and U2R in this study (Section V) means classical-ML IDS should never be the sole defence against these attack families; layered detection (host-based, behavioural, payload-inspection) is required to compensate.

Bias and dataset provenance. NSL-KDD's underlying traffic is drawn from the DARPA 1998 simulation, which represents US-military network conditions of nearly three decades ago. A detector trained on this data inherits its blind spots: encrypted traffic is absent, modern application-layer attacks (HTTP-based exploits, modern phishing infrastructure) are not represented, and the geographic and linguistic diversity of more recent traffic is missing. A Nepal-deployed system trained on NSL-KDD alone would be likely to under-detect locally relevant attack patterns. A responsible production deployment would require fine-tuning on representative local traffic, not direct transplantation.

Legal context. In Nepal, network monitoring is governed by the Electronic Transactions Act 2008 and the Privacy Act 2018, which require lawful basis and proportionality for processing communications data. International deployments would additionally need to consider GDPR Article 6 (lawful basis) and Article 32 (security of processing). The use of synthetic data (SMOTE) for training does not by itself raise additional legal exposure because synthetic samples do not contain real user information, but the underlying training data does, and so retention and access controls apply to it.

Professional responsibility. Reporting only cross-validation scores — the convenient but misleading number that many NSL-KDD papers quote — misrepresents what a model can do once deployed. Professional integrity requires that practitioners separate distribution-fitting performance from generalisation performance and communicate the difference to non-technical decision makers. This paper takes that requirement seriously, which is why both numbers are reported and why the gap between them is discussed at length.

VII. Conclusions

We compared three classical machine learning algorithms — Logistic Regression, Random Forest, and XGBoost — on the full five-class NSL-KDD intrusion detection problem under identical preprocessing and evaluation conditions. XGBoost achieved the best test accuracy (78.66%), the best macro F1 (0.6337), and the best detection of rare attack types, with R2L and U2R remaining hard for all three models. We also documented a roughly 22-percentage-point gap between cross-validation and held-out test scores, which we trace to the deliberate inclusion of unseen attack variants in NSL-KDD's test split.

The work has clear limitations. We used a single benchmark dataset, restricted to classical algorithms, and applied only one resampling strategy. Most importantly, NSL-KDD's underlying traffic is from 1998; modern attack patterns (HTTP-based exploits, modern DDoS, encrypted-channel exfiltration) are not represented. The strongest validation of an NSL-KDD-trained detector would be to apply it to a more recent benchmark such as CICIDS2017 [7] and observe how well the model generalises. We leave this validation as future work along with cost-sensitive learning that explicitly penalises missed rare-attack detections more than false alarms on Normal traffic.

Acknowledgements

The author thanks Softwarica College of IT & E-Commerce and Coventry University for the academic environment in which this work was carried out, and the maintainers of the NSL-KDD dataset for providing a benchmark that has supported two decades of intrusion detection research.

Reproducibility and Supplementary Material

The complete source code, LaTeX sources, generated figures, per-model result files, and a pedagogical Jupyter notebook (45 cells with embedded outputs) reproducing every experiment are publicly available at:

Source code (GitHub):
https://github.com/SudipAdh/
NETWORK-INTRUSION-DETECTION

Demo screencast (Google Drive):
https://drive.google.com/file/d/
12qWAM443JASkQnvJphx7Uv3OxtabJP6s/view?usp=sharing

The repository includes a README with run instructions, dataset download links, and a one-command reproduction script. All randomness is seeded with `RANDOM_SEED = 42`; results in this paper are reproducible bit-for-bit on a Python 3.13 environment with the package versions pinned in `requirements.txt`. The screencast is a short walkthrough of the Jupyter notebook running end-to-end on the author's machine and is intended for reviewers who prefer not to set up the environment locally.

Appendix

Appendix A: Per-Fold Cross-Validation Evidence

Table IV reports the weighted F1 score on each fold of the 5-fold stratified cross-validation, after per-fold SMOTE resampling, for each of the three algorithms. The very small standard deviation (between 0.0002 and 0.0015) indicates that fold-to-fold variation is negligible compared to the gap with held-out test performance.

TABLE IV: Per-fold weighted F1 on the NSL-KDD training set (5-fold stratified CV with SMOTE applied per fold).

Algorithm	F1	F2	F3	F4	F5	Mean	Std
LR	0.9777	0.9766	0.9750	0.9735	0.9753	0.9756	0.0013
RF	0.9988	0.9987	0.9989	0.9987	0.9984	0.9987	0.0002
XGBoost	0.9990	0.9991	0.9994	0.9991	0.9988	0.9991	0.0002

Appendix B: Full Per-Class Test Results

Tables V–VII present the complete precision, recall, and F1 scores per class on the held-out KDDTest+ split for each of the three models.

TABLE V: Logistic Regression — per-class results on KDDTest+.

	Prec.	Recall	F1	Support
Normal	0.7191	0.9162	0.8058	9,711
DoS	0.9785	0.8379	0.9028	7,458
Probe	0.7094	0.7613	0.7344	2,421
R2L	0.7179	0.1877	0.2976	2,887
U2R	0.0762	0.4925	0.1320	67

TABLE VI: Random Forest — per-class results on KDDTest+.

	Prec.	Recall	F1	Support
Normal	0.6507	0.9735	0.7800	9,711
DoS	0.9604	0.7540	0.8447	7,458
Probe	0.8464	0.6919	0.7614	2,421
R2L	0.9581	0.0554	0.1048	2,887
U2R	0.6154	0.1194	0.2000	67

TABLE VII: XGBoost — per-class results on KDDTest+.

	Prec.	Recall	F1	Support
Normal	0.6936	0.9719	0.8095	9,711
DoS	0.9641	0.8073	0.8788	7,458
Probe	0.8209	0.7518	0.7848	2,421
R2L	0.9818	0.1496	0.2597	2,887
U2R	0.6471	0.3284	0.4356	67

References

[1] KDD Cup 1999 Data. UCI Machine Learning Repository. Available: http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html

[2] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pp. 1–6, 2009.

[3] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 1, e4150, 2021.

[4] L. Dhanabal and S. P. Shantharajah, "A study on NSL-KDD dataset for intrusion detection system based on classification algorithms," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 4, no. 6, pp. 446–452, 2015.

[5] B. Ingre and A. Yadav, "Performance analysis of NSL-KDD dataset using ANN," in *Proc. International Conference on Signal Processing and Communication Engineering Systems (SPACES)*, pp. 92–96, 2015.

[6] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve Bayes feature embedding," *Computers & Security*, vol. 103, p. 102158, 2021.

[7] R. Panigrahi and S. Borah, "A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems," *International Journal of Engineering & Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[11] "Anomaly-based intrusion detection on benchmark datasets for network security: a comprehensive evaluation," *Scientific Reports*, vol. 16, 2026.

[12] "Advanced IDS: a comparative study of datasets and machine learning algorithms for network flow-based intrusion detection systems," *Applied Intelligence*, 2025.